

Computer Architecture Assignment 3

National Tsing Hua University
Department of Computer Science

Exercise

This exercise is to practice procedure call, recursive call, and bit manipulation operations.

Question

Write a MIPS assembly program for the following C program.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 void mat_mul(unsigned a[2][2], unsigned b[2][2], unsigned res[2][2]) {
5     unsigned tmp[2][2] = {0};
6     for (unsigned i = 0; i < 2; ++i) {
7         for (unsigned j = 0; j < 2; ++j) {
8             for (unsigned k = 0; k < 2; ++k) {
9                 tmp[i][j] += a[i][k] * b[k][j];
10            }
11        }
12    }
13    memcpy(res, tmp, sizeof(tmp));
14 }
15
16 void mat_fast_power_recursive(unsigned base[2][2], unsigned exp, unsigned res[2][2])
17 {
18     if (exp == 0) {
19         // Base case: return identity matrix
20         res[0][0] = res[1][1] = 1;
21         res[0][1] = res[1][0] = 0;
22         return;
23     }
24     if (exp == 1) {
25         // Base case: return the base matrix
26         memcpy(res, base, sizeof(unsigned) * 4);
27         return;
28     }
29     unsigned temp[2][2];
30     mat_fast_power_recursive(base, exp / 2, temp);
31     mat_mul(temp, temp, res);
32     if (exp % 2 == 1) {
33         unsigned temp2[2][2];
34         memcpy(temp2, res, sizeof(temp2));
35         mat_mul(temp2, base, res);
36     }
37 }
38
39 // Function to count the number of 1s in the binary representation
40 unsigned count_bits(unsigned n) {
41     if (n == 0) return 0;
42     return count_bits(n >> 1) + (n & 1);
43 }
44
45 int main() {
```

```

45 unsigned trans[2][2] = {{1, 1}, {1, 0}};
46 int n; // Changed to int to handle -1
47
48 printf("Enter n values (enter -1 to exit):\n");
49
50 while (1) {
51     printf("Input n: ");
52     scanf("%d", &n);
53
54     // Check for exit condition
55     if (n == -1) {
56         printf("Program terminated.\n");
57         break;
58     }
59
60     // Check for valid input
61     if (n < 0) {
62         printf("Invalid input! Please enter a non-negative integer or -1 to exit\n");
63         continue;
64     }
65
66     unsigned result[2][2];
67     mat_fast_power_recursive(trans, (unsigned)n, result);
68
69     unsigned fib_n = result[0][1];
70     unsigned bit_count = count_bits(fib_n);
71
72     printf("fib[%d] = %u\n", n, fib_n);
73     printf("1s count in 32 LSBs of fib[%d]: %u\n\n", n, bit_count);
74 }
75
76 return 0;
77 }

```

Input Constraints

- $0 \leq n < 2^{31}$ for valid Fibonacci calculation
- $n = -1$ to exit the program
- $n < -1$ will display an error message

Requirements

- You must use procedure (function) call to implement all functions (mat_fast_power_recursive, count_bits).
- Your program should support continuous input until -1 is entered.
- Your program should count and display the number of 1s in the binary representation of each Fibonacci number.
- Your program should handle invalid inputs gracefully.
- Your program should terminate normally (the output should show "– program is finished running –").

Output Format Example

Enter n values (enter -1 to exit):

Input n: 10

fib[10] = 55

1s count in 32 LSBs of fib[10]: 5

Input n: 20

fib[20] = 6765

1s count in 32 LSBs of fib[20]: 8

Input n: 30

fib[30] = 832040

1s count in 32 LSBs of fib[30]: 8

Input n: -1

Program terminated.

-- program is finished running --

Submission

- Submit 1 assembly program
- Name your assembly program with your student ID, for example: "arch_hw3_123456789.asm"
- Use eeclass (<https://eeclass.nthu.edu.tw/>) to submit your program

Grading Criteria

- Correctness: 70%
- Bit counting implementation: 10%
- Comment in program: 10%
- Output format: 10%

Contact Information

- Teaching Assistant: Wuqian Tang
- Email: wqtang@cs.nthu.edu.tw
- Office: Room 228, EECS Building

Deadline: 2025.10.06 23:30

Appendix

A Matrix Exponentiation for Fibonacci Numbers

The principle behind using matrix exponentiation to calculate Fibonacci numbers is based on the linear recurrence relation of the Fibonacci sequence and the properties of matrix multiplication.

A.1 Fibonacci Recurrence Relation

The Fibonacci sequence is defined by the recurrence relation:

$$F_n = F_{n-1} + F_{n-2} \quad (1)$$

with initial conditions $F_0 = 0$ and $F_1 = 1$.

A.2 Matrix Representation

We can represent this recurrence relation using a 2x2 matrix:

$$\begin{bmatrix} F_{n+1} \\ F_n \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} \quad (2)$$

Let's call the matrix $T = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$.

A.3 Matrix Exponentiation

By applying this transformation n times, we get:

$$\begin{bmatrix} F_{n+1} \\ F_n \end{bmatrix} = T^n \begin{bmatrix} F_1 \\ F_0 \end{bmatrix} = T^n \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad (3)$$

Therefore, to find the n th Fibonacci number, we need to calculate T^n and then multiply it by $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$.

A.4 Fast Exponentiation

The key to efficiency is using the fast exponentiation algorithm (also known as exponentiation by squaring) to calculate T^n . This reduces the time complexity from $O(n)$ to $O(\log n)$.

The algorithm works as follows:

- If n is even, $T^n = (T^{n/2})^2$
- If n is odd, $T^n = T \cdot (T^{(n-1)/2})^2$

This recursive approach allows us to calculate high Fibonacci numbers efficiently, making it possible to compute Fibonacci numbers for very large n values.

B Bit Counting Algorithm

The bit counting function counts the number of 1s in the 32-bit binary representation of a number. This is also known as calculating the Hamming weight or population count.

B.1 Algorithm Description

The algorithm uses a simple iterative approach:

1. Initialize a counter to 0
2. While the number is not zero:
 - Check if the least significant bit is 1 using bitwise AND with 1
 - Add the result to the counter
 - Right shift the number by 1 position to examine the next bit
3. Return the counter value

B.2 Example

For the Fibonacci number $F_{10} = 55$:

- Binary representation: $55_{10} = 110111_2$
- Number of 1s: 5

This bit counting operation has various applications in computer science, including cryptography, error detection codes, and optimization algorithms.